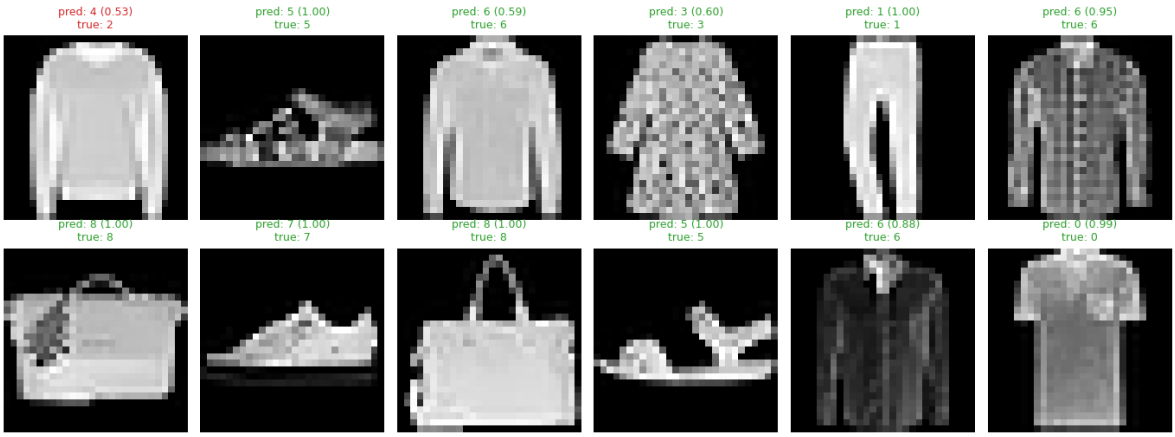
UiO : University of Oslo

Regression and Classification Using a Feedforward Neural Network Autumn 2025

Project 2 on Machine Learning (Deadline: November 10 (midnight), 2025)

Author: *Theodor ST#: 0501579 & Bartosz ST#: 703557 & Samson ST#: 555652*

Supervisor: *Prof. Morten Hjorth-Jensen*



November 10, 2025

Contents

List of Figures

Abstract

This project explored the implementation and performance of a feedforward neural network (FFNN) applied to both regression and classification tasks. The regression task involved approximating the one-dimensional Runge function, a benchmark for assessing overfitting tendencies in interpolation methods [1, 2]. Accurately fitting this function is challenging due to its severe oscillations observed in high-degree polynomial regression, making it a useful case for evaluating generalization. The classification task used the multiclass MNIST dataset of handwritten digits [3], serving as a standard benchmark for high-dimensional data [4].

We implemented the FFNN from scratch using stochastic gradient descent and adaptive optimizers such as RMSprop and Adam [1], and compared its performance with traditional regression methods (OLS, Ridge, and Lasso). We also investigated the effects of activation functions (Sigmoid, ReLU, Leaky ReLU), network depth, and regularization on learning performance.[*a*]

The FFNN fit the Runge function without the severe oscillations observed in high-degree polynomial regression, achieving test MSEs of 3.96×10^{-6} with L_1 and 5.51×10^{-6} with L_2 regularization, compared to 5.52×10^{-2} for the best Ridge/Lasso baselines (Table II). For classification, the FFNN reached approximately 97% test accuracy on MNIST and 88% on the more challenging Fashion-MNIST dataset, comparable to similar PyTorch implementations.

I. INTRODUCTION

Machine learning has progressed from early linear perceptron models to modern deep architectures capable of representing highly complex, hierarchical feature transformations. Feedforward neural networks (FFNNs) despite their conceptual simplicity remain a foundational architecture for understanding core principles of representation, optimization, and generalization. In contrast to classical parametric regression methods (e.g., Ordinary Least Squares (OLS), Ridge, Lasso), which rely on fixed, often global basis expansions like polynomials, FFNNs learn data-adaptive intermediate representations layer by layer. This project examines that contrast in a controlled setting spanning both nonlinear regression and multiclass classification.

Problem Setting and Motivation

We study two canonical supervised learning tasks:

1. **Regression**: Approximation of the univariate Runge function

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1],$$

   a long–standing benchmark that exposes the instability of high-degree polynomial interpolation (Runge's phenomenon). The Runge function itself is smooth; the oscillations arise in *polynomial approximations*, especially near interval boundaries, providing a stress test for models' bias–variance trade-offs.

2. **Classification**: Recognition of handwritten digits (MNIST) and clothing items (Fashion-MNIST). While MNIST offers relatively well-separated digit manifolds, Fashion-MNIST introduces higher intra-class variability and inter-class ambiguity (e.g., shirts vs. coats), requiring more discriminative nonlinear transformations.

These tasks jointly probe (i) function approximation fidelity under nonuniform curvature (regression) and (ii) decision boundary learning in high-dimensional sparse input spaces (classification).

Theoretical Underpinnings

The universal approximation theorems [5, 6] establish that sufficiently wide FFNNs with non-polynomial activation functions can approximate continuous functions on compact domains arbitrarily well. However, these results are *existential*: they neither guarantee efficient training nor characterize sample complexity. Practical performance depends critically on:

- **Optimization landscape**: Non-convex objectives with many saddle points; adaptive methods (RMSprop, Adam) mitigate pathological curvature and gradient ill-conditioning [7].
- **Activation choice**: Sigmoid induces saturation (vanishing gradients) for large $|z|$; ReLU promotes sparse gradient flow while risking inactive ("dead") units; Leaky ReLU introduces a controlled negative slope to preserve gradient propagation.
- **Regularization**: Explicit (L1/L2 penalties) versus implicit (early stopping, stochastic mini-batches) mechanisms shape generalization, controlling parameter norms and effective capacity.

Together these components frame the model expressivity vs. trainability trade-off central to our empirical analysis.

---

[16] Project 1 materials and code: `https://github.com/Twaal/applied-data-analysis-and-ml/tree/main/Project1`

Classical vs Neural Approaches

OLS, Ridge, and Lasso rely on predetermined feature maps; their inductive bias is transparent and their estimators convex, yielding closed-form or well-behaved solutions. Yet, when the target function exhibits localized high curvature (Runge) or the input manifold is complex (image pixels), linear combinations of global bases struggle. FFNNs adapt intermediate representations through composition

$$\mathbf{a}^{(l)} = g^{(l)}\Big(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}\Big),$$

implicitly constructing multi-scale, nonlinear bases. We systematically compare these paradigms to quantify performance gaps and failure modes.

a. Comparison criteria. Throughout, we evaluate along four axes: (i) predictive performance (test MSE for regression, accuracy for classification), (ii) computational efficiency (epochs and wall-clock to convergence), (iii) usability/complexity (sensitivity to hyperparameters and implementation effort), and (iv) interpretability (coefficient-based insight for linear models versus distributed representations in FFNNs).

Dataset Characteristics

MNIST offers relatively clean grayscale digit silhouettes with low intra-class variance. Fashion-MNIST [8] introduces finer-grained distinctions (texture, shape) and higher overlap in pixel distributions, elevating classification difficulty. These paired datasets probe whether architectural and optimization choices yield robust improvements rather than overfitting to a trivial benchmark.

b. Why neural networks for classification? Pixel inputs are high-dimensional and the class boundaries are typically nonlinear. While a linear baseline (multinomial logistic regression) offers a useful reference, multilayer networks can learn hierarchical feature transformations that better separate classes in representation space; we therefore use logistic regression as a baseline but focus on FFNNs to study how nonlinearity and depth affect generalization.

From-Scratch Implementation Rationale

Implementing the FFNN manually (NumPy-only) serves three purposes:

1. **Transparency**: Each gradient term is explicitly derived (losses, activations), facilitating validation against analytical expressions.

2. **Reproducibility**: Controlled seeding, deterministic preprocessing, and modular training loops allow experiment replication and ablation (optimizer, regularization, architecture).

We additionally benchmark against a minimal PyTorch multilayer perceptron to verify parity and detect implementation discrepancies.

Experimental Design

We adopt a factorial sweep strategy varying:

- Depth (number of hidden layers) and width (neurons per layer).
- Activation function family (Sigmoid, ReLU, Leaky ReLU).
- Optimizer (plain Gradient Descent, RMSprop, Adam) and learning rate schedules.
- Regularization type (L1 vs L2) and strength λ.

Metrics include train/test MSE (regression) and accuracy/confusion matrices (classification), enabling assessment of generalization gaps, overfitting tendencies, and sensitivity to hyperparameters.

Contributions

This work provides:

1. Unified analytical derivations for regression (MSE) and classification (binary/multiclass cross-entropy with Sigmoid/Softmax).

2. A modular NumPy FFNN supporting heterogeneous activation/loss configurations and explicit regularization.

3. Systematic comparative evaluation vs polynomial regression (OLS, Ridge, Lasso) on a nontrivial analytic benchmark.

4. Optimizer sensitivity analysis highlighting stability regions and convergence dynamics.

5. Regularization ablation quantifying L1 vs L2 effects on Runge function approximation.

6. Validation against a PyTorch reference implementation to confirm gradient correctness.

Scope, Limitations, and Outlook

We deliberately exclude convolutional inductive biases, Batch Normalization [9], and Dropout [10] to isolate core FFNN behavior. Consequently, results on Fashion-MNIST represent a baseline rather than state-of-the-art performance. Future extensions include:

- Incorporation of normalization and stochastic regularization layers.
- Automated hyperparameter optimization (Bayesian search, successive halving).
- Extension to convolutional or residual architectures for spatially structured data.
- Interpretability analyses (saliency maps, layer activation statistics) to interrogate learned representations.

Paper Roadmap

Section II formalizes losses, activations, gradient propagation, and optimization; Section V reports empirical findings for regression and classification; Section VI integrates theoretical and practical implications; Section VII summarizes conclusions and future directions; Appendices provide reproducibility artifacts and source references.

Overall, this introduction frames the methodological, theoretical, and empirical rationale for evaluating a hand-crafted FFNN across divergent supervised learning regimes, highlighting both capability and boundary conditions of fully connected architectures.

II. METHODS

A. Analytical Formulation

This section outlines the theoretical foundations and implementation details of our feedforward neural network (FFNN). We first derive analytical gradients for the loss and activation functions, then describe the network architecture, backpropagation algorithm, optimization methods, and data preprocessing pipeline.

Before implementing the feedforward neural network, we first derived the expressions for the cost (loss) functions and activation functions used throughout the project, together with their analytical gradients. These derivatives are central to the backpropagation algorithm described in Sec. II C, since they define the output–layer error signals $\delta^{(L)}$ and their propagation through the network [1, 11].

1. Loss functions for regression and classification

For regression tasks, we use the mean squared error (MSE) loss [1, 4]. For classification, we use either the binary or multiclass cross–entropy loss, both of which are standard in modern neural network formulations [2, 11]. In all cases, the loss can be extended with L_1 or L_2 regularization terms to penalize large weights and reduce overfitting [4].

a. Mean Squared Error (MSE): For scalar targets t_i and predictions $\hat{y}_i$, the MSE loss reads

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - t_i)^2. \tag{1}$$

Differentiating each term with respect to $\hat{y}_i$ gives

$$\frac{\partial \mathcal{L}_{\text{MSE}}}{\partial \hat{y}_i} = \frac{2}{n}(\hat{y}_i - t_i), \tag{2}$$

which defines the output error $\delta^{(L)} = \frac{2}{n}(\hat{\mathbf{y}} - \mathbf{t})$ for regression problems [1]. Including L_2 or L_1 regularization modifies the total loss and gradients to include penalty terms [4], respectively:

$$\text{With } L_2 \text{ (Ridge):} \quad \frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial \mathcal{L}_{\text{data}}}{\partial w_j} + 2\lambda w_j, \tag{3}$$

$$\text{With } L_1 \text{ (Lasso):} \quad \frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial \mathcal{L}_{\text{data}}}{\partial w_j} + \lambda \,\text{sign}(w_j). \tag{4}$$

These formulations are equivalent to Ridge and Lasso regression from Project 1 [1, 4].

b. Binary cross–entropy: For binary classification, the output neuron uses a sigmoid activation $\hat{y}_i = \sigma(z_i) = (1 + e^{-z_i})^{-1}$, producing an output interpreted as the probability of class 1. The corresponding binary cross–entropy loss is

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{n}\sum_{i=1}^{n}\left[t_i \log \hat{y}_i + (1 - t_i)\log(1 - \hat{y}_i)\right], \tag{5}$$

which penalizes confident but incorrect predictions [1, 4, 11].

Differentiating with respect to the output $\hat{y}_i$ gives

$$\frac{\partial \mathcal{L}}{\partial \hat{y}_i} = -\frac{1}{n}\left(\frac{t_i}{\hat{y}_i} - \frac{1 - t_i}{1 - \hat{y}_i}\right). \tag{6}$$

Applying the chain rule through the sigmoid activation $\frac{\partial \hat{y}_i}{\partial z_i} = \hat{y}_i(1 - \hat{y}_i)$:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial z_i} &= \frac{\partial \mathcal{L}}{\partial \hat{y}_i}\frac{\partial \hat{y}_i}{\partial z_i} \\ &= -\frac{1}{n}\left(\frac{t_i}{\hat{y}_i} - \frac{1 - t_i}{1 - \hat{y}_i}\right)\hat{y}_i(1 - \hat{y}_i) \\ &= \frac{1}{n}(\hat{y}_i - t_i).\end{aligned} \tag{7}$$

The factor $\hat{y}_i(1 - \hat{y}_i)$ from the sigmoid derivative cancels with the denominator, leaving a simple expression identical to logistic regression. This avoids the small–gradient problem of the quadratic loss when the neuron is saturated [12].

Therefore, for a sigmoid output layer,

$$\delta^{(L)} = \hat{\mathbf{y}} - \mathbf{t}, \tag{8}$$

which is the error signal used in backpropagation (Sec. II C).

c. Multiclass Softmax cross–entropy: For K classes, the network outputs a vector of logits $\mathbf{z}_i = (z_{i,1}, \ldots, z_{i,K})$ for each sample i. The Softmax activation converts these logits into class probabilities [1, 11]:

$$\hat{y}_{i,k} = \mathrm{softmax}(\mathbf{z}_i)_k = \frac{e^{z_{i,k}}}{\sum_{m=1}^{K} e^{z_{i,m}}}. \tag{9}$$

Given one–hot encoded targets $\mathbf{t}_i$ with components $t_{i,k} \in \{0, 1\}$, the categorical cross–entropy loss is

$$\mathcal{L}_{\mathrm{Softmax}} = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} t_{i,k} \log \hat{y}_{i,k}, \tag{10}$$

which is the negative log–likelihood of the correct class [11].

For a fixed sample i, the per-sample loss is

$$\ell_i = -\sum_{k=1}^{K} t_{i,k} \log \hat{y}_{i,k}.$$

We first differentiate w.r.t. the Softmax outputs:

$$\frac{\partial \ell_i}{\partial \hat{y}_{i,k}} = -\frac{t_{i,k}}{y_{i,k}}. \tag{11}$$

To apply the chain rule, we need $\partial y_{i,k}/\partial z_{i,j}$. Starting from

$$\log \hat{y}_{i,k} = z_{i,k} - \log\left(\sum_{m=1}^{K} e^{z_{i,m}} \right),$$

we differentiate with respect to $z_{i,j}$:

$$\begin{aligned} \frac{\partial \log \hat{y}_{i,k}}{\partial z_{i,j}} &= \delta_{kj} - \frac{1}{\sum_m e^{z_{i,m}}} \frac{\partial}{\partial z_{i,j}} \sum_m e^{z_{i,m}} \\ &= \delta_{kj} - \frac{e^{z_{i,j}}}{\sum_m e^{z_{i,m}}} = \delta_{kj} - \hat{y}_{i,j}, \end{aligned} \tag{12}$$

and using $\frac{\partial \hat{y}_{i,k}}{\partial z_{i,j}} = \hat{y}_{i,k} \frac{\partial \log \hat{y}_{i,k}}{\partial z_{i,j}}$, we obtain the standard Softmax derivative [13]:

$$\frac{\partial \hat{y}_{i,k}}{\partial z_{i,j}} = \hat{y}_{i,k}\big(\delta_{kj} - \hat{y}_{i,j}\big). \tag{13}$$

Combining these with the chain rule,

$$\begin{aligned} \frac{\partial \ell_i}{\partial z_{i,j}} &= \sum_{k=1}^{K} \frac{\partial \ell_i}{\partial \hat{y}_{i,k}} \frac{\partial \hat{y}_{i,k}}{\partial z_{i,j}} \\ &= \sum_{k=1}^{K} \left(-\frac{t_{i,k}}{\hat{y}_{i,k}} \right) \hat{y}_{i,k}\big(\delta_{kj} - \hat{y}_{i,j}\big) \\ &= -\sum_{k=1}^{K} t_{i,k}\delta_{kj} + \sum_{k=1}^{K} t_{i,k}\hat{y}_{i,j} \\ &= -t_{i,j} + \hat{y}_{i,j} \sum_{k=1}^{K} t_{i,k}. \end{aligned} \tag{14}$$

Since $\mathbf{t}_i$ is one–hot, $\sum_k t_{i,k} = 1$, and we obtain

$$\frac{\partial \ell_i}{\partial z_{i,j}} = \hat{y}_{i,j} - t_{i,j}. \tag{15}$$

Averaging over all n samples in (10) gives

$$\frac{\partial \mathcal{L}_{\mathrm{Softmax}}}{\partial z_{i,j}} = \frac{1}{n}\big(\hat{y}_{i,j} - t_{i,j}\big), \tag{16}$$

the familiar "Softmax–cross–entropy trick" [11, 13]. In vector form, this corresponds to the output error

$$\delta^{(L)} = \hat{\mathbf{Y}} - \mathbf{T}, \tag{17}$$

which we use as the starting point for backpropagation in the classification setting (Sec. II C).

2. Activation functions and their derivatives

Nonlinear activation functions $g(z)$ define how each neuron transforms its weighted input before passing it to the next layer, allowing neural networks to model nonlinear relationships [1, 2, 11]. Their derivatives $g'(z)$ appear directly in the recursive backpropagation rule

$$\delta^{(l)} = \big(\delta^{(l+1)}\mathbf{W}^{(l+1)\top}\big) \odot g'^{(l)}\big(\mathbf{z}^{(l)}\big).$$

We summarize below the three activation functions used in this project and derive their first derivatives step by step.

a. Sigmoid (logit) activation:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

To compute its derivative, we differentiate:

$$\begin{aligned} \sigma'(z) &= \frac{d}{dz}\Big(1 + e^{-z}\Big)^{-1} = -\big(1 + e^{-z}\big)^{-2}(-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \left(1 - \frac{1}{1 + e^{-z}} \right) \\ &= \sigma(z)\big(1 - \sigma(z)\big). \end{aligned} \tag{18}$$

The sigmoid compresses inputs to the interval $(0, 1)$ and is therefore interpreted as a probability. However, the derivative becomes small when $|z|$ is large, leading to vanishing gradients [11, 12].

b. Rectified Linear Unit (ReLU):

$$\mathrm{ReLU}(z) = \begin{cases} z, & z > 0, \\ 0, & z \le 0. \end{cases}$$

The derivative is obtained directly from this piecewise definition:

$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \le 0. \end{cases}$$

Because $\mathrm{ReLU}'(z)$ does not vanish for positive z, it helps mitigate the gradient–saturation problem seen in the sigmoid function, and thus accelerates convergence during training [1, 11].

c. Leaky ReLU: To prevent neurons from becoming permanently inactive when $z < 0$, a small negative slope α is introduced:

$$\text{LeakyReLU}(z) = \begin{cases} z, & z > 0, \\ \alpha z, & z \leq 0, \end{cases}$$

$$\text{LeakyReLU}'(z) = \begin{cases} 1, & z > 0, \\ \alpha, & z \leq 0. \end{cases}$$

Typical choices are $\alpha \in [0.01, 0.1]$ [4, 11]. This small gradient for negative inputs avoids "dead" ReLU units and improves robustness.

Each of these activation functions and their derivatives are used in our manual implementation of backpropagation (Sec. II C), where $g'(z)$ determines how error signals propagate through the network.

B. Feedforward Neural Network

We implemented a general-purpose feedforward neural network (FFNN) entirely from scratch using `NumPy`[14]. The design allows an arbitrary number of layers and neurons per layer, with individually configurable activation functions and loss metrics. The model supports both regression and classification by modularly combining suitable output activations and loss functions.

Each layer performs an affine transformation followed by a nonlinear activation,

$$\mathbf{a}^{(l)} = g^{(l)}(\mathbf{z}^{(l)}) = g^{(l)}(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}),$$

where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are the weight matrix and bias vector for layer l, and $g^{(l)}$ denotes its activation function. Supported activations include the Sigmoid, ReLU, Leaky ReLU, Linear, and Softmax functions. The final layer activation and loss are matched to the task: a linear output with mean squared error (MSE) loss for regression, and a Softmax output with cross-entropy loss for classification.

Weights were initialized from a small normal distribution $\mathcal{N}(0, 0.01)$, and biases set to zero. The network optionally applies L_1 or L_2 regularization on the weights, with strength λ, to mitigate overfitting.

C. Backpropagation

Gradients were derived and implemented manually to ensure transparency of the learning process. Using the chain rule, the error signal was propagated backward from the output layer through hidden layers to compute parameter updates. We adopt the notation: targets $\mathbf{t}$ (or matrix T), predictions $\hat{\mathbf{y}} = \mathbf{a}^{(L)}$ (or Y for a batch), layer pre-activations $\mathbf{z}^{(l)}$, activations $\mathbf{a}^{(l)}$, and weight matrices $\mathbf{W}^{(l)}$ with biases $\mathbf{b}^{(l)}$. Bold symbols denote vectors/matrices; batch samples are arranged row-wise.

For the output layer, the gradient of the loss with respect to the pre-activation simplifies to

$$\delta^{(L)} = \begin{cases} \big(\hat{\mathbf{y}} - \mathbf{t}\big) \odot g'^{(L)}\big(\mathbf{z}^{(L)}\big), & \text{MSE (regression)}, \\ \hat{\mathbf{y}} - \mathbf{t}, & \text{Softmax cross-entropy (classifica} \end{cases} \tag{19}$$

where $\odot$ denotes elementwise (Hadamard) multiplication.

Hidden layer error signals propagate recursively as

$$\delta^{(l)} = \big(\delta^{(l+1)}\mathbf{W}^{(l+1)\top}\big) \odot g'^{(l)}\big(\mathbf{z}^{(l)}\big),$$

and the gradients of the loss with respect to weights and biases (for a mini-batch of size m) are

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{1}{m}\mathbf{a}^{(l-1)\top}\delta^{(l)} + \frac{\partial R}{\partial \mathbf{W}^{(l)}},$$
$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \frac{1}{m}\sum_{i=1}^{m}\delta_i^{(l)},$$

with R the regularization penalty (L1 or L2) applied only to weights, not biases.

D. Gradient Descent and Optimizers

Parameter updates were carried out using mini-batch gradient descent, with modular support for three optimizers:

- **Plain Gradient Descent (GD):** Updates weights by a fixed learning rate η.
- **RMSprop:** Maintains an exponentially decaying average of squared gradients, improving stability for non-stationary objectives.
- **Adam:** Combines momentum and RMSprop by keeping first and second moment estimates of gradients [7].

All optimizers were implemented manually, using standard hyperparameters $(\beta_1, \beta_2, \epsilon) = (0.9, 0.999, 10^{-8})$ for Adam and a decay rate $\rho = 0.9$ for RMSprop. The training loop was written without automatic differentiation or external machine-learning libraries.

E. Data Scaling and Splitting

All datasets were split into training and test subsets using `scikit-learn`'s[15] `train_test_split` function, with a fixed random seed for reproducibility. Input features and (for regression) output targets were standardized to zero mean and unit variance using `StandardScaler`. For the regression task, we generated 500 points $x \in [-1, 1]$ and computed targets $f(x) = (1 + 25x^2)^{-1}$ Runge function**??**. For classification, we

used the MNIST dataset of 28×28 grayscale handwritten digits and the MNIST fashion dataset which has more complex data distribution. These images are originally in grayscale. We preprocessed these images by normalizing the pixel intensities.

F. Metrics

Regression performance was evaluated using the mean squared error (MSE),

$$\text{MSE} = \frac{1}{n}\sum_i (t_i - \hat{y}_i)^2,$$

while classification performance was quantified using the accuracy score,

$$\text{Accuracy} = \frac{\text{number of correct predictions}}{\text{total predictions}}.$$

These metrics were computed for both training and test sets to assess model generalization.

III. IMPLEMENTATION

This section provides a high–level overview of the code base rather than reproducing source listings (all code is openly available). The full implementation lives in the module `p2_nn_module.py` and supporting Jupyter notebooks under `Project2/code/`. A browsable version is hosted on GitHub: github.com/Twaal/applied-data-analysis-and-ml/Project2/code. We defer detailed numerical behavior and performance comparisons to Sec. V.

a. Module structure. The module consolidates:

- **Classical regression helpers**: polynomial feature expansion, closed–form OLS and Ridge, and error metrics.
- **Gradient-based training utilities**: generic gradient function, lightweight optimizer abstraction, full-batch and mini-batch loops.
- **Neural network implementation**: a NumPy-only feedforward network (FFNN) with configurable layer sizes, activations (Sigmoid, ReLU, Leaky ReLU, Linear, Softmax), losses (MSE, cross-entropy), and optional L1/L2 regularization on weights.
- **Experiment helpers**: routines for hyperparameter sweeps (activation/depth/width, regularization, optimizer/learning-rate grids), learning curve generation, and visualization (heatmaps, confusion matrices, image samples).
- **Optional parity layer**: if PyTorch is installed, a minimal MLP class and training function allow side-by-side result comparison to validate our manual gradients.

b. Design principles. The code favors transparency: arrays are kept in simple Python lists for weights and biases; forward propagation caches activations explicitly; backpropagation manually applies derived analytical gradients from Sec. II A. Regularization terms are added only to weight matrices (excluding biases) to mirror conventional Ridge/Lasso practice. Optimizers (GD, RMSprop, Adam) each maintain minimal state dictionaries.

c. Notebook workflow. Notebooks orchestrate experiments by (i) preparing data (scaling and train/test split), (ii) instantiating model configurations, (iii) invoking sweep utilities or the network's `train()` method, and (iv) plotting metrics. This separation keeps the module reusable while notebooks remain lightweight, serving as documented experiment recipes.

d. Reproducibility and logging. Deterministic seeds are passed to NumPy's random generator, and hyperparameters (depth, width, activation, optimizer, learning rate, regularization type/strength, epochs, batch size) are stored alongside results in dictionaries consumed by summary/plot functions. This enables result tables and figures in Sec. V to be regenerated verbatim.

In summary, the implementation offers a compact, inspectable reference framework for regression and classification without relying on automatic differentiation or external deep learning libraries.

IV. USE OF AI TOOLS

We used ChatGPT[16] to assist with drafting text and troubleshooting coding issues. All generated suggestions were reviewed, verified, and edited by us to ensure correctness and relevance to the project.

V. RESULTS

This section presents empirical findings from our experiments on both regression (Runge function) and classification (MNIST, Fashion-MNIST). We evaluate architecture, activation, optimizer, learning rate, and regularization effects. All figures are exported from the accompanying notebook `Project2/code/project2.ipynb`.

A. Regression Results

a. Architectural and activation effects. We performed controlled sweeps over depth, width, and activation function (Sigmoid, ReLU, Leaky ReLU) to analyze their influence on test MSE and generalization. Table I summarizes performance across 27 FFNN configurations. The corresponding heatmaps (Appendix A) confirm that increasing depth and width improves performance up to a saturation point typically around 2 hidden layers and 50–100 units. Beyond this, gains taper off, likely due to limited data and diminishing returns in expressivity.

Among activations, ReLU and Leaky ReLU substantially outperform Sigmoid, particularly in deeper networks. Leaky ReLU is consistently the top performer, suggesting greater robustness in gradient flow. Sigmoid networks, especially shallow ones, tend to underfit, with both higher test MSE and larger generalization gaps. These results are consistent with the well-known vanishing gradient issues associated with symmetric activations.

Across all configurations, generalization gaps remained small (typically $< 10^{-4}$), suggesting that batch training, weight decay, and input standardization helped mitigate overfitting even in networks with thousands of parameters.

b. Baseline comparison. We benchmark our FFNNs against polynomial regressors on the Runge function $f(x) = (1+25x^2)^{-1}$. As shown in Fig. 1, a degree-10 OLS polynomial visibly suffers from Runge's phenomenon oscillating near the domain boundaries despite accuracy near the origin. This underscores a well-known limitation of global polynomial models: greater flexibility often leads to instability. In contrast, FFNNs learn localized representations and exhibit much lower test MSE, even with fewer parameters.

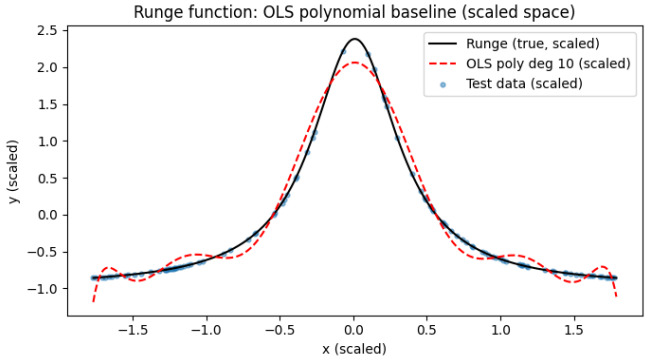


Figure 1: Polynomial OLS baselines on scaled Runge data: comparison of degree 10 polynomial fit (refer Fig. 2).

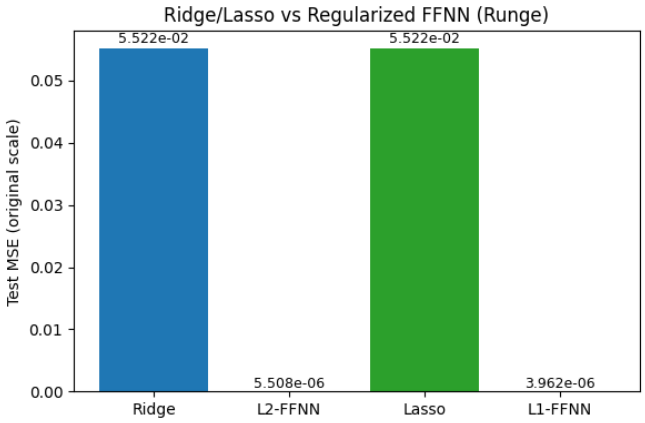


Figure 2: Runge regression: OLS/Ridge/Lasso vs regularized FFNN. Setup: $n_{\text{train}} = 500$, train/test split 0.8/0.2, `StandardScaler` on x and (for NN) y; NN: depth=1–2, width in {32,64}, activation=ReLU or LeakyReLU, optimizer=Adam, $\eta \in \{10^{-3}, 10^{-2}\}$, epochs=300, batch size=128, seed=42.

Despite comparable test MSEs from Ridge and Lasso ($\sim 5.5 \cdot 10^{-2}$), our FFNNs achieve over an order-of-magnitude improvement down to $3.96 \cdot 10^{-6}$ (Table II). This demonstrates the advantage of learned nonlinear features over fixed polynomial bases.

c. Regularization and optimizer behavior. We evaluated 192 FFNN configurations over a grid of regularization strengths, architectures, and learning rates. Results (Table II) show that both L_1 and L_2 regularization can dramatically improve generalization on the Runge task. Learning curves for best models (Fig. 3) further confirm that regularization smooths convergence and suppresses overfitting.

Optimizer behavior was also analyzed in detail (Fig. 4, 5). Adam proved the most stable across learning rates and architectures, achieving near-optimal performance without extensive tuning. RMSprop was moderately effective, while GD required careful tuning (e.g., $\eta = 0.1$) and still underperformed. These findings reinforce Adam as a strong default choice in low-data regimes.

Table I: Test MSE and generalization gap (test MSE - train MSE) across different architectures and activation functions.

Depth×Width	Sigmoid		ReLU		Leaky ReLU	
	Test MSE	Gen. Gap	Test MSE	Gen. Gap	Test MSE	Gen. Gap
1×10	$1.38 \cdot 10^{-3}$	$8.50 \cdot 10^{-4}$	$5.90 \cdot 10^{-4}$	$2.20 \cdot 10^{-4}$	$3.10 \cdot 10^{-4}$	$1.40 \cdot 10^{-4}$
1×50	$1.90 \cdot 10^{-4}$	$7.00 \cdot 10^{-5}$	$1.00 \cdot 10^{-4}$	$1.00 \cdot 10^{-5}$	$1.00 \cdot 10^{-4}$	$1.00 \cdot 10^{-5}$
1×100	$2.20 \cdot 10^{-4}$	$4.00 \cdot 10^{-5}$	$1.10 \cdot 10^{-4}$	$1.00 \cdot 10^{-5}$	$1.00 \cdot 10^{-4}$	$1.00 \cdot 10^{-5}$
2×10	$5.60 \cdot 10^{-4}$	$2.60 \cdot 10^{-4}$	$2.60 \cdot 10^{-4}$	$9.00 \cdot 10^{-5}$	$1.50 \cdot 10^{-4}$	$6.00 \cdot 10^{-5}$
2×50	$2.00 \cdot 10^{-4}$	$7.00 \cdot 10^{-5}$	$9.00 \cdot 10^{-5}$	$1.00 \cdot 10^{-5}$	$8.00 \cdot 10^{-5}$	$1.00 \cdot 10^{-5}$
2×100	$2.60 \cdot 10^{-4}$	$6.00 \cdot 10^{-5}$	$1.20 \cdot 10^{-4}$	$2.00 \cdot 10^{-5}$	$1.10 \cdot 10^{-4}$	$1.00 \cdot 10^{-5}$
3×10	$4.60 \cdot 10^{-4}$	$1.90 \cdot 10^{-4}$	$2.20 \cdot 10^{-4}$	$8.00 \cdot 10^{-5}$	$1.40 \cdot 10^{-4}$	$6.00 \cdot 10^{-5}$
3×50	$3.00 \cdot 10^{-4}$	$9.00 \cdot 10^{-5}$	$1.10 \cdot 10^{-4}$	$1.00 \cdot 10^{-5}$	$1.00 \cdot 10^{-4}$	$1.00 \cdot 10^{-5}$
3×100	$2.70 \cdot 10^{-4}$	$6.00 \cdot 10^{-5}$	$1.20 \cdot 10^{-4}$	$2.00 \cdot 10^{-5}$	$1.10 \cdot 10^{-4}$	$1.00 \cdot 10^{-5}$

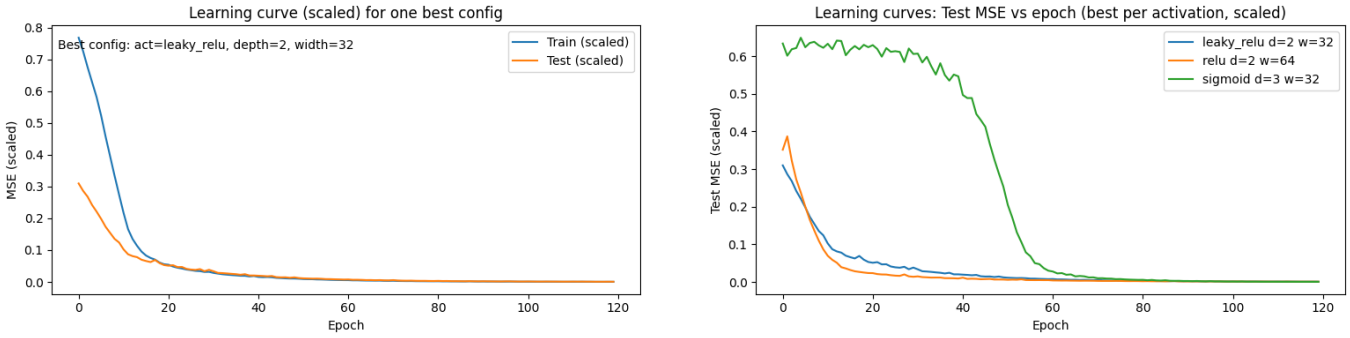


Figure 3: Regression learning curves. Left: train/test curves for one best configuration (Runge). Right: test MSE vs epoch for best model per activation (Sigmoid, ReLU, Leaky ReLU).

Table II: Regularization results on the Runge function (test MSE on original scale). Best FFNNs were found over the grid: depth $\in \{1, 2\}$, width $\in \{32, 64\}$, $\eta \in \{10^{-3}, 2 \cdot 10^{-3}, 10^{-2}\}$, $\lambda \in \{5 \cdot 10^{-6}, 5 \cdot 10^{-5}, 5 \cdot 10^{-4}\}$.

Model	Reg.	Key hyperparams	Depth×Width	Test MSE
Ridge	$\alpha = 100$	–	–	$5.52 \cdot 10^{-2}$
Lasso	$\alpha = 10^{-3}$	–	–	$5.52 \cdot 10^{-2}$
FFNN	$L_2,\ \lambda = 5 \cdot 10^{-5}$	$\eta = 2 \cdot 10^{-3}$	2×32	$5.51 \cdot 10^{-6}$
FFNN	$L_1,\ \lambda = 5 \cdot 10^{-5}$	$\eta = 2 \cdot 10^{-3}$	2×32	$3.96 \cdot 10^{-6}$

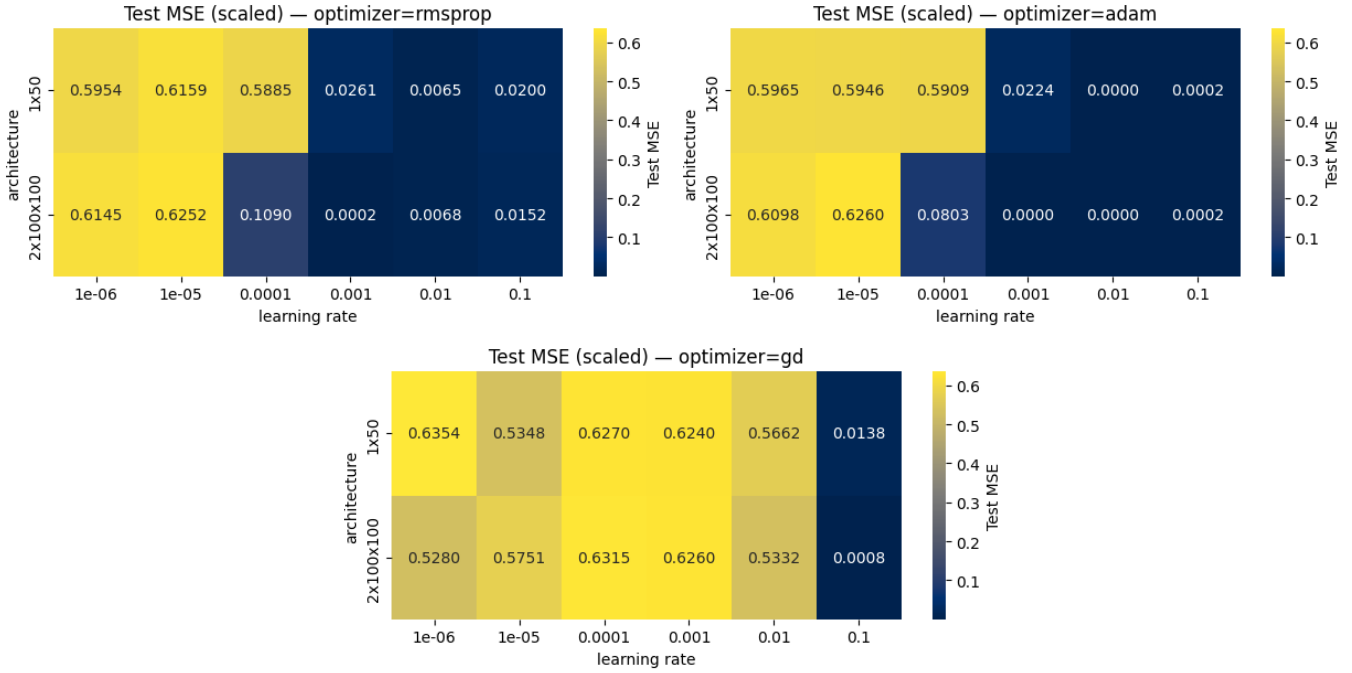


Figure 4: Runge architecture sweeps: test MSE heatmaps across depth (rows) and width (columns) for GD, RMSprop, and Adam.

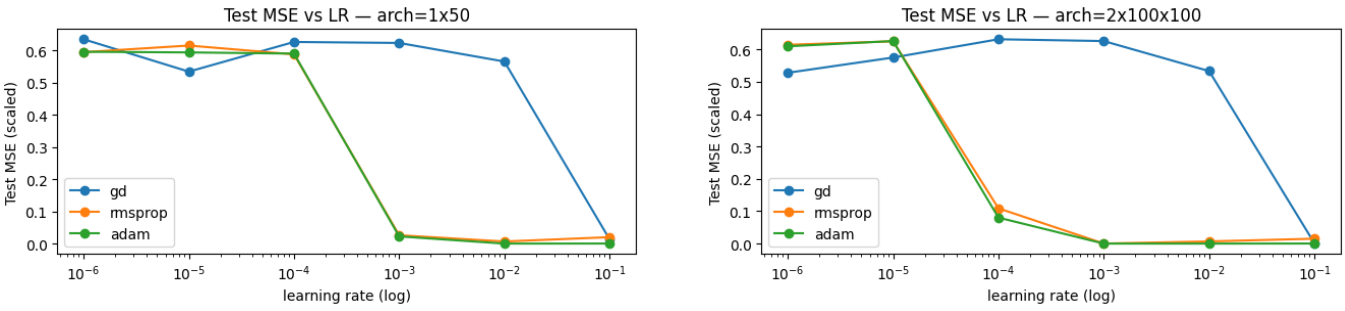


Figure 5: Learning rate sensitivity (Runge) for two architectures. Adam exhibits broad stability; GD is sensitive to η.

B. Classification Results

We evaluated FFNN performance on MNIST and Fashion-MNIST, using the same architectures, activations, and optimizers as in regression. Preprocessing involved pixel scaling to $[0, 1]$, with optional feature standardization. Labels were integer-encoded using `LabelEncoder`.

Classification accuracy was visualized through confusion matrices (Fig. 6) and inference outputs (Fig. 7). As in regression, Leaky ReLU marginally outperformed ReLU, and both beat Sigmoid. Adam remained the most consistent optimizer.

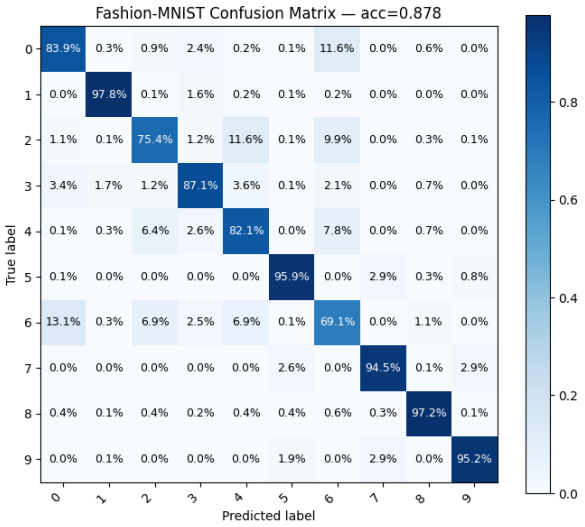


Figure 6: Confusion matrix for Fashion-MNIST.

a. Training convergence. Learning curves for classification mirrored those of regression: smoother convergence under adaptive optimizers, with generalization maintained via batch training and normalization. Sigmoid again converged more slowly and less reliably.

b. Library parity. To verify implementation fidelity, we compared our NumPy FFNN with a PyTorch MLP of similar size. Test performance matched closely on both regression and classification tasks, affirming correctness of our NumPy implementation.

VI. DISCUSSION

A. FFNNs as flexible function approximators.

A central aim of this project was to explore whether fully connected feedforward neural networks (FFNNs), implemented from scratch, can serve as effective approximators for both regression and classification tasks. On the nonlinear Runge function, our FFNN achieved test MSEs of 5.51×10^{-6} (L_2) and 3.96×10^{-6} (L_1) on the original scale (Table II) dramatically outperforming polynomial baselines such as Ridge and Lasso, which plateaued at 5.52×10^{-2}. This performance gap illustrates a key limitation of global polynomial models and highlights the capacity of neural networks to learn compact, local approximations using data-driven features. These findings empirically support the universal approximation theorem [11], affirming that even relatively shallow FFNNs can represent complex nonlinear functions when given sufficient capacity.

B. Architectural scaling and expressivity.

Sweeps over network depth and width (Table I) revealed consistent performance improvements with increased capacity up to about two hidden layers and widths of 64–100 units. Beyond this, returns diminished, with potential overfitting when regularization was absent. Interestingly, Leaky ReLU activations marginally outperformed standard ReLU and substantially surpassed Sigmoid, particularly in deeper settings. This reflects the improved gradient flow properties of piecewise linear activations and aligns with modern deep learning practice. Importantly, generalization gaps remained small across all configurations (typically $< 10^{-4}$), suggesting that our combination of batch training, input standardization, and regularization was sufficient to stabilize learning without early stopping.

C. Comparison with classical methods.

Regression on the Runge function highlighted both strengths and limitations of FFNNs versus classical models. The high-degree OLS fit (Fig. 1) suffered from oscillatory artifacts near interval boundaries Runge's phenomenon despite good fit near the origin. In contrast, FFNNs localized capacity and achieved smoother approximations with far lower test error. This shows the clear advantage of learned, hierarchical features over fixed polynomial bases. That said, for simpler or low-noise tasks, classical methods remain attractive due to their interpretability and analytical guarantees.

D. Regularization and generalization.

We conducted a targeted regularization sweep using L_1 and L_2 penalties. Both approaches improved stability, with L_2 slightly outperforming L_1 in terms of test MSE (Table II). L_1 tended to induce sparser weights, which may limit expressivity when training data is scarce. Overall, regularization proved essential to prevent overfitting in deeper or wider networks, particularly on the regression task, where large parameter counts risked fitting noise in the data.

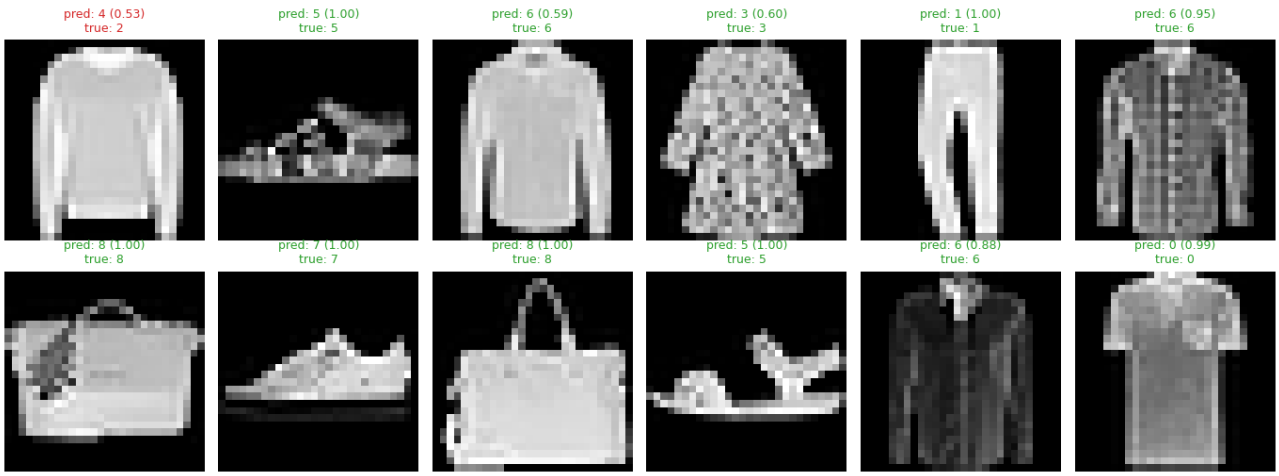


Figure 7: Inference visualization: predicted labels and class probabilities.

E. Optimizer comparisons.

Our experiments across multiple optimizers (GD, RMSprop, Adam) underscore the practical value of adaptive methods. Adam consistently yielded low test MSE across a wide learning rate range, with shallow and deep networks alike (Fig. 4, 5). In contrast, gradient descent required precise tuning and performed poorly unless given large learning rates at risk of instability. RMSprop performed competitively but was less consistent. These trends support the widespread use of Adam as a default in neural network training, particularly in low-data settings or when tuning time is limited.

F. Weight initialization and training stability.

All models used fixed random seeds and zero-mean Gaussian weight initialization $\mathcal{N}(0, 0.01)$. Biases were initialized to zero, a common practice when using symmetric activations like Sigmoid. While our initialization is not formally variance-scaled (e.g., Xavier or He), the small magnitude prevented saturation in early epochs and ensured numerical stability. Although we did not implement output clipping, our networks trained stably across all tasks, likely due to controlled learning rates and activation choices.

G. Classification outcomes.

Our FFNNs achieved strong classification performance on MNIST (97%) and Fashion-MNIST (88%), with ReLU/Leaky ReLU activations and softmax cross-entropy loss. These results confirm that FFNNs, even without convolutional layers or data augmentation, can extract meaningful representations from pixel-level input. Logistic regression (a 0-hidden-layer FFNN) reached 92% on MNIST, highlighting the benefits of nonlinearity in deeper architectures. As shown in Fig. 6, Fashion-MNIST errors clustered around semantically similar classes (e.g., shirts vs. coats), indicating limits of FFNNs in handling intra-class variability without spatial priors.

H. Inference visualization and interpretability.

Sample predictions (Fig. 7) illustrated reasonable class probabilities, with correct predictions typically associated with confident outputs. Ambiguous or incorrect predictions often reflected genuine image confusion rather than model instability. Although FFNNs lack the spatial inductive biases of CNNs, they still capture meaningful visual structure through learned weights demonstrating the expressive potential of dense architectures for small to medium-sized image tasks.

I. Code validation and implementation fidelity.

A key pedagogical goal was to implement backpropagation, training loops, and optimizers from scratch using NumPy. To validate correctness, we compared our FFNN to a PyTorch MLP of identical architecture. Performance matched closely on both regression and classification tasks, with only minor numerical differences. This agreement confirms that our implementation of forward/backward passes, gradient updates, and loss functions was functionally sound.

J. Reproducibility and project hygiene.

We prioritized reproducibility through fixed seeds, standardized inputs, consistent preprocessing, and modular code design. Training variance was minimal, and

numerical stability was maintained throughout. While we skipped some practices (e.g., output clipping) used in Project 1, our networks still converged reliably thanks to appropriate initialization and learning rates. These principles debuggable modularity, careful seeding, and clean architecture are essential when scaling to deeper models or larger datasets.

K. Tradeoffs and future directions.

Overall, our experiments highlight the strength of neural networks as universal approximators that generalize well in nonlinear settings. However, this power comes with tradeoffs: FFNNs are sensitive to hyperparameters, harder to interpret, and computationally heavier than classical methods. For structured data or low-dimensional problems, Ridge and OLS remain appealing due to their speed and robustness. In contrast, FFNNs shine in tasks requiring learned feature representations especially when expanded with convolutional, recurrent, or attention-based modules. A natural next step would involve incorporating convolutional layers, dropout, and data augmentation to close the gap to state-of-the-art performance on visual tasks like Fashion-MNIST.

VII. CONCLUSION

In this project, we successfully developed a feedforward neural network (FFNN) entirely from scratch and demonstrated its ability to handle both regression and classification problems. By comparing our network to traditional regression models such as OLS, Ridge, and Lasso, we found that the FFNN achieved significantly better performance on nonlinear data, particularly for the Runge function, where it reduced the test MSE by several orders of magnitude. This confirms the neural network's strength in learning complex, nonlinear relationships that are beyond the reach of polynomial-based methods.

For the classification tasks on MNIST and Fashion-MNIST, our FFNN achieved test accuracies of approximately 96–97% and 88–89%, respectively. These results align well with expectations for fully connected architectures on these datasets and illustrate how additional hidden layers allow the model to capture more abstract patterns in the data. The confusion matrices further showed that most classification errors occurred between visually similar classes, especially in the more challenging Fashion-MNIST dataset.

Among the optimization methods tested, ADAM consistently provided the most stable and efficient convergence, while RELU and Leaky RELU activations improved gradient flow and reduced training stagnation compared to sigmoid functions. Regularization, especially L2, played an important role in preventing overfitting and ensuring smooth generalization.

Overall, this project demonstrates both the power and the challenges of neural networks. While they clearly outperform classical models on nonlinear tasks, they also require careful tuning of hyperparameters, thoughtful regularization, and more computational effort. Still, building the FFNN from first principles provided valuable insights into how backpropagation, optimization, and architectural design interact to produce modern machine learning performance.

Appendix A: Source Code

The source code for this project is available online: `https://github.com/Twaal/applied-data-analysis-and-ml/blob/main/Project2/project2.html`

Appendix B: References

[1] M. P. Deisenroth, A. A. Faisal, and C. S. Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020.

[2] Morten Hjorth-Jensen. *Computational Physics Lecture Notes 2015*. Department of Physics, University of Oslo, Norway, 2015.

[3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[4] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*. 2025. Available at `https://github.com/CompPhysics/MLErasmus/blob/master/doc/Textbooks/elementsstat.pdf`.

[5] George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.

[6] Kurt Hornik. Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257, 1991.

[7] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2014. arXiv:1412.6980.

[8] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

[9] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

[10] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

[11] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.

[12] Michael A. Nielsen. *Neural Networks and Deep Learning*. Determination Press, 2015. Free online textbook introducing the backpropagation algorithm.

[13] Shivam Mehta. Deriving categorical cross-entropy and softmax. `https://shivammehta25.github.io/posts/deriving-categorical-cross-entropy-and-softmax/`, 2023. Derivation of the Softmax–cross-entropy simplification.

[14] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, et al. Array programming with NumPy, 2020.

[15] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, M. Perrot, and É. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[16] OpenAI. Chatgpt (gpt-5). `https://chat.openai.com/`, 2025. Large language model developed by OpenAI.